DAY 2

Module 4

From Clarified Spec to Reviewable Plan

Turn a what-and-why spec into a reviewable how, before any code changes.

What You'll Learn

1. Translate a clarified spec into a technical plan
2. Use a plan/preview mode to review before any files change
3. Decompose a plan into small, independently verifiable tasks

FROM SPEC TO PLAN

Where Implementation Detail Belongs

The plan is where architecture choices, constraints, and non-functional requirements belong.

It's the bridge between what and why (the spec) and how (the code).

REVIEW BEFORE CODE

Plan/Preview Mode and Task Decomposition

A reviewable plan is produced before any file changes are made: a deliberate checkpoint.

The team reviews and edits the plan, including disagreeing with an architecture or scope decision, before an agent touches code.

Break the approved plan into small, independently verifiable, agent-executable tasks. Each task should map back to a specific requirement.

WHY IT MATTERS

A plan you didn't review is a decision you already made.

Every architecture and scope choice in the plan becomes real the moment implement runs.

Catching a disagreement here costs a conversation. Catching it after implementation costs a rewrite.

APPLYING THIS WITH

GitHub Spec Kit

SPEC KIT

Plan, Then Tasks

`/speckit.plan`

Generates design artifacts from the spec, given tech stack, architecture, and constraints as arguments.

`/speckit.tasks`

Generates a dependency-ordered tasks.md: Setup, Foundational, one phase per user story, and a final Polish phase. Tasks are marked for parallel execution where possible.

APPLYING THIS WITH

OpenSpec

OPENSPEC

Design and Tasks, Together or Staged

The plan: design.md is the plan equivalent, "the how: technical approach and architecture decisions," generated alongside the proposal.

The tasks: tasks.md, an implementation checklist with checkboxes, is generated in the same step by default.

```
/opsx:continue
```

Creates one artifact at a time for teams that want to review each before the next is generated, instead of the default all-at-once /opsx:propose or /opsx:ff.

Module 4 at a Glance

	GitHub Spec Kit	OpenSpec
Plan	/speckit.plan (separate command)	design.md (part of propose/ff)
Tasks	/speckit.tasks (separate command)	tasks.md (same step as design)
Staged review	Natural: each command is a checkpoint	/opsx:continue, one artifact at a time

Key Takeaways

- A plan you haven't reviewed is a plan you're implicitly approving.
- Small tasks are easier to verify and easier to isolate when something breaks.
- Both frameworks support staged review: Spec Kit through separate commands, OpenSpec through /opsx:continue.

HANDS-ON NEXT

Generate a plan from your project's spec and review it line by line as a team. Edit at least one decision you disagree with, then decompose the approved plan into tasks and confirm each one maps back to a specific requirement.

Next: Module 5: Quality Gates Before Implementation